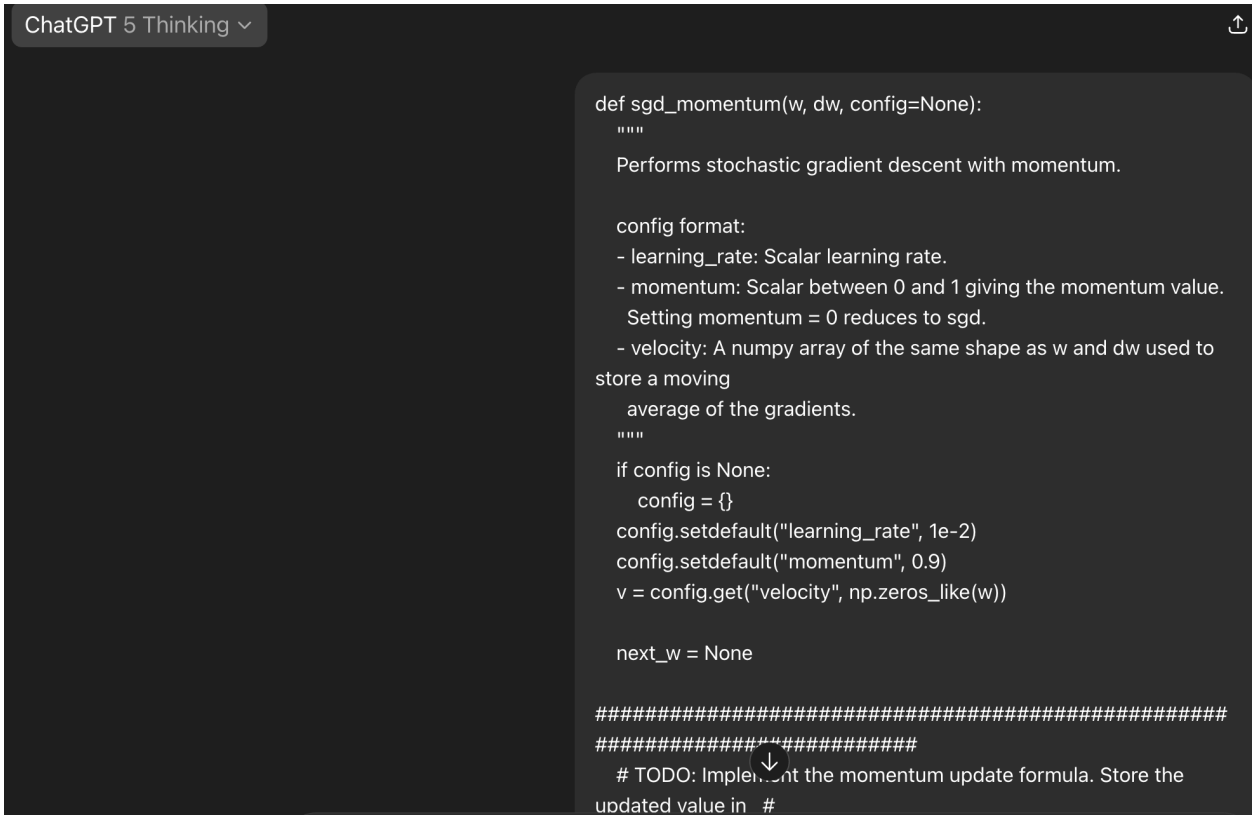
Special Participation B: Homework 2 with GPT-5

Executive Summary: In this assignment, I used GPT-5 with thinking to assist with the coding portion of Homework 2 by employing various strategies to provide the model with full context (markdown, code skeletons, and images). For most coding tasks, the model produced correct solutions immediately, but there were some mistakes. When it made mistakes, it corrected them once I provided additional context or error messages. For the written portion of the coding homework, I did not notice any issues. For visualization problems, I prompted GPT-5 with images of notebook outputs or slider settings. It consistently gave accurate interpretations tied directly to features visible in the images. Across the assignment, its errors were minor and easily fixed.

3. Coding Question: Initialization and Optimizers

The first coding portion I had the opportunity to use GPT-5 on was for the momentum implementation.

I attempt to one shot the problem with just the function skeleton.



```
    # the next_w variable. You should also use and update the velocity
v.     #
    ###########################################################
    ###########################
    pass  # TODO

    ###########################################################
    ###########################
    #                   END OF YOUR CODE                    #
    ###########################################################
    ###########################
    config["velocity"] = v

    return next_w, config
```

Here's the standard momentum update (classical, not Nesterov):

python Copy code

```python
import numpy as np

def sgd_momentum(w, dw, config=None):
    """
    Performs stochastic gradient descent with momentum.

    config format:
    - learning_rate: Scalar learning rate.
    - momentum: Scalar between 0 and 1 giving the momentum value.
      Setting momentum = 0 reduces to sgd.
    - velocity: A numpy array of the same shape as w used to store a moving
      average of the gradients.
    """
    if config is None:
        config = {}
    lr = config.setdefault("learning_rate", 1e-2)
    mu = config.setdefault("momentum", 0.9)

    v = config.get("velocity", np.zeros_like(w))
```

```
    # Momentum update
    v = mu * v - lr * dw
    next_w = w + v

    # Store velocity back in config
    config["velocity"] = v

    return next_w, config
```

Copy code

Update rule: `v ← μ v − η ∇w` and `w ← w + v`.

2/2

Let's try this code.

```
print ('next_w error: ', rel_error(next_w, expected_next_w))
print ('velocity error: ', rel_error(expected_velocity, config['v
```

```
next_w error:  1.0
velocity error:  1.0
```

It does not one-shot this problem and gives me high error values. Looking closely, the moment update rule GPT-5 gives me is different from the formulation provided.

```
# Momentum update
v = mu * v - lr * dw
next_w = w + v
```

Our formulation of momentum:

The momentum update rule for a weight matrix w is:

$$\begin{cases} v_{dw}^{t} = m * v_{dw}^{(t-1)} + dw \\ w = w - \alpha v_{dw}^{t} \end{cases} \quad (3)$$

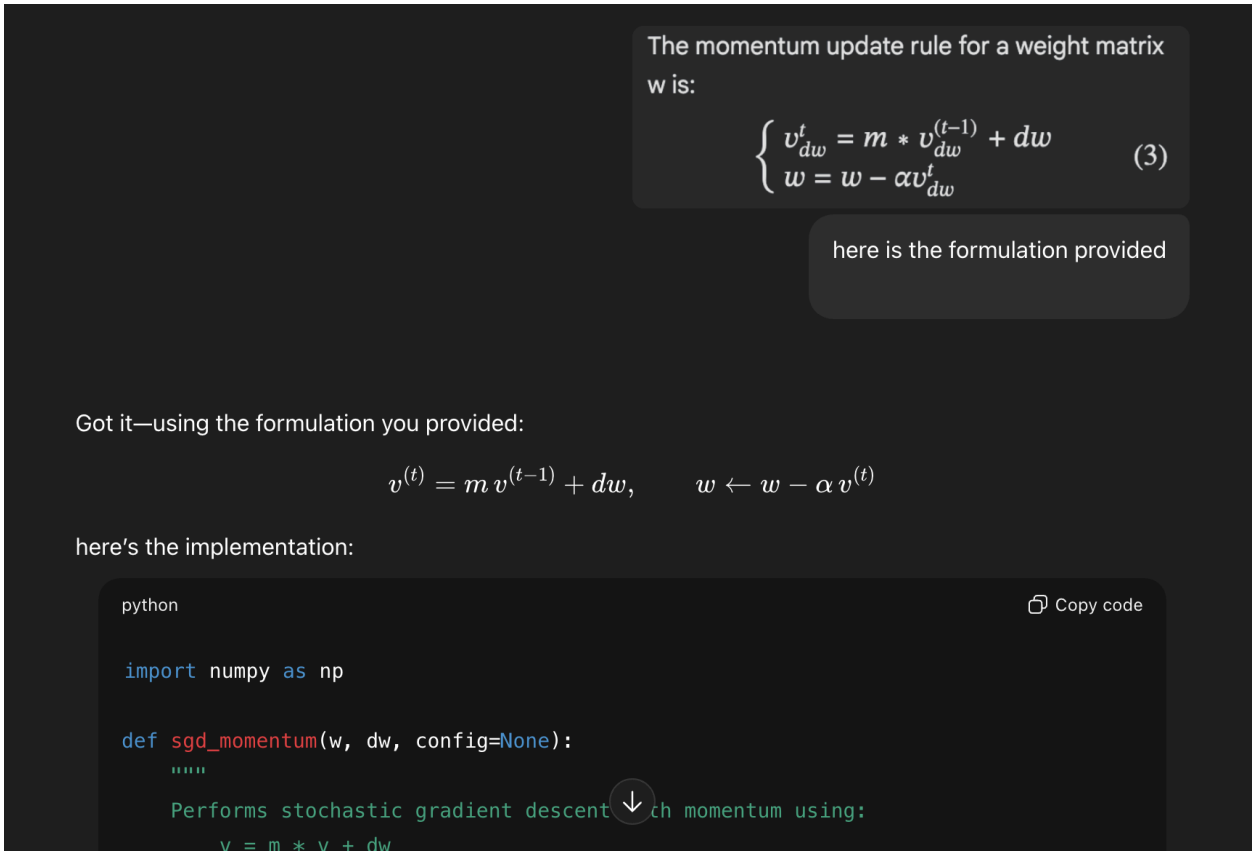
I searched online and there are actually two equivalent forms of expressing momentum: https://ai.stackexchange.com/questions/25152/how-are-these-equations-of-sgd-with-momentum-equivalent

$$v_t = \rho v_{t-1} + \nabla f(x_{t-1})$$
$$x_t = x_{t-1} - \alpha v_t,$$

$$v_t = \rho v_{t-1} + \alpha \nabla f(x_{t-1})$$
$$x_t = x_{t-1} - v_t,$$

The first one is our version, and the second one resembles the code GPT-5 gave me, albeit with opposite signs.

I add some additional context, the version of the update rule we use in this class, and see if it can adapt its answer.

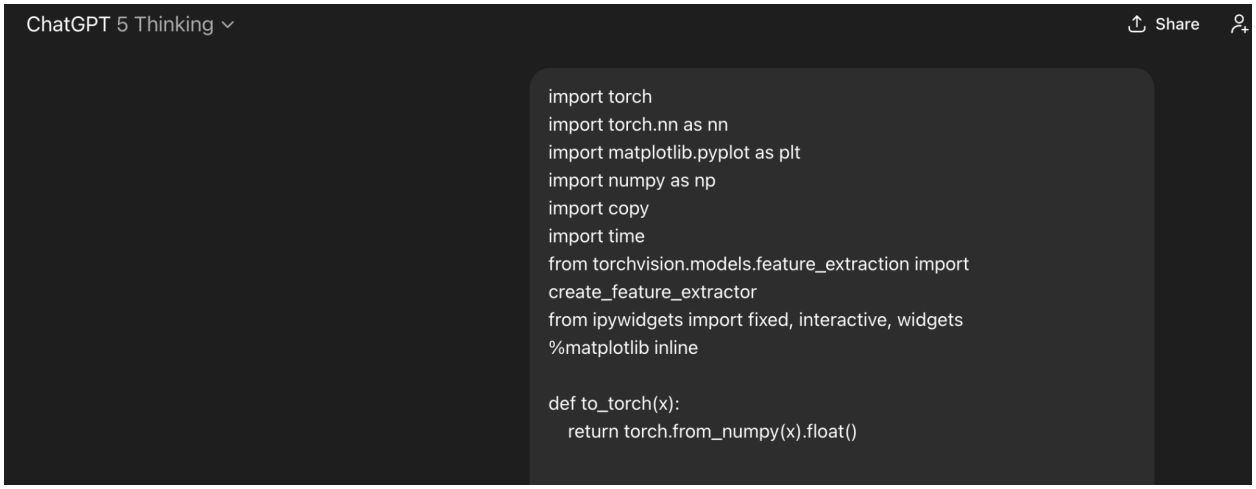


```
    w = w - lr * v
"""
if config is None:
    config = {}
lr = config.setdefault("learning_rate", 1e-2)
mu = config.setdefault("momentum", 0.9)

v = config.get("velocity", np.zeros_like(w))

# Update velocity and weights per the given formulation
v = mu * v + dw
next_w = w - lr * v

# Save updated velocity
config["velocity"] = v
return next_w, config
```

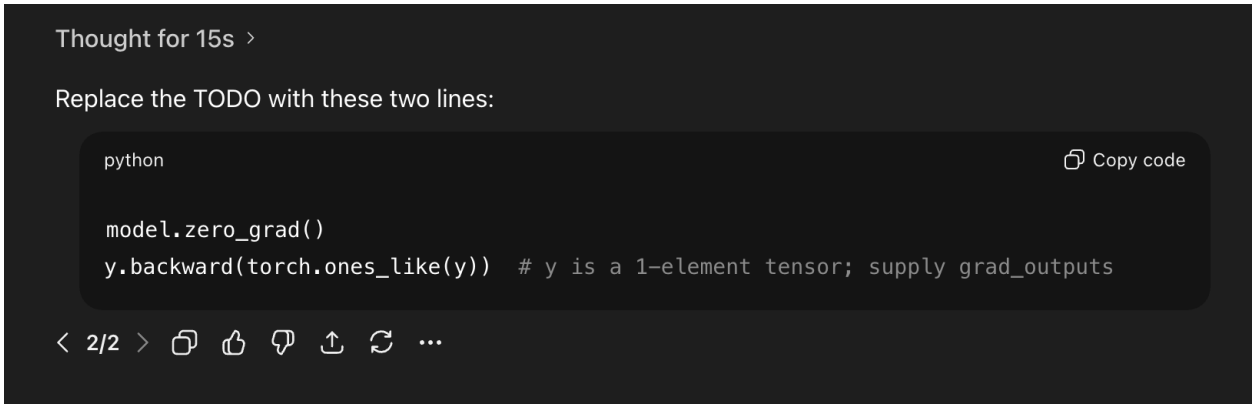
I use this new function and it works perfectly.
Ideally, I should have provided more context from the start, but it was interesting to see it guess what conventional context the problem is in.

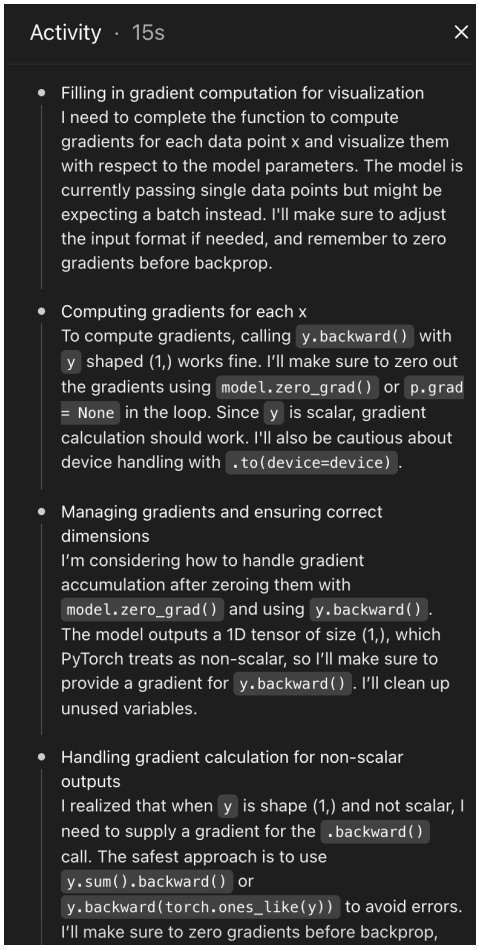
4. Visualizing features from local linearization of neural nets

Since it's tricky to use GPT-5 with a colab notebook, I copied every cell into the text input prior to my query cell. For brevity, I omit most of it but this is what the beginning looks like:



It gives me very brief snippet of code to paste at the YOUR CODE HERE section.





The code works and the chain of thought seems sound as well. The model's solution is unremarkable here since I expected it to do this easy problem on the first try.

I do the same approach for the next problem, leveraging the existing context from the previous part of the conversation as well.

(b) SVD for feature matrix

During training, we can imagine that we have a generalized
linear model with a feature matrix corresponding to the linearized
features corresponding to each learnable parameter. We know from
our analysis of gradient descent, that the singular values and
singular vectors corresponding to this feature matrix are important.

Use the SVD of this feature matrix to plot both the singular values and visualize the “principle
features” that correspond to the d-dimensional singular vectors multiplied by all the features
corresponding to the parameters

Thought for a few seconds ›

Use the same two backprop lines as in (a), then do a compact SVD and project features:

python Copy code

```
# backprop (same as part a)
model.zero_grad()
y.backward(torch.ones_like(y))
```

python Copy code

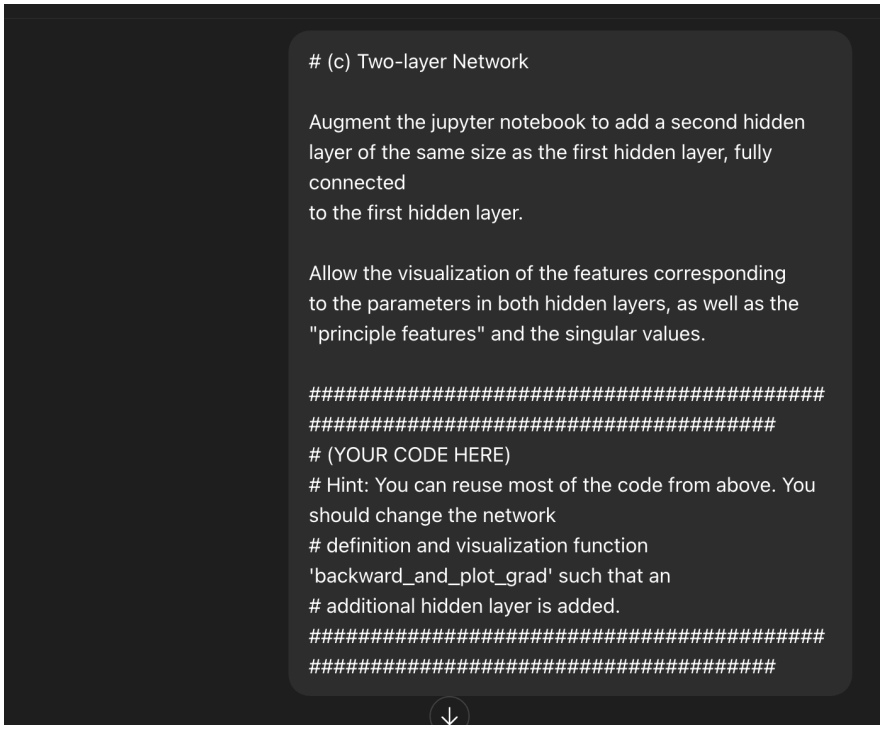
```
# SVD
u, s, vh = np.linalg.svd(feature_matrix, full_matrices=False)
```

python Copy code

```
# principal features (project features onto right-singular vectors)
princple_feature = feature_matrix @ vh.T
```

Something funny to note is that it uses “princple_feature” as a variable name. This is a strange name since it conflates principal and principle and also misspells it too. Other than that, the code is perfect.

For the final problem, there is no skeleton code so I expect this one to be the most interesting. I take the same approach again, prompting with the notebook markdown as context and leveraging the conversation history as context.



Thought for 6s ›

Here's a clean drop-in cell to extend your notebook with a 2-hidden-layer ReLU net (same width), and to visualize gradients for **both** hidden layers, plus singular values and principal features. It reuses your helpers and only adds what's needed.

python Copy code

```python
# === (c) Two-layer Network: architecture, training, and visualization

# 1) Build a 2-hidden-layer ReLU net (width -> width -> 1)
def build_two_layer_net(width):
    net = nn.Sequential(
        nn.Linear(1, width),   # layer 0
        nn.ReLU(),
        nn.Linear(width, width),  # layer 2 (second hidden layer)
        nn.ReLU(),
        nn.Linear(width, 1)    # layer 4 (output)
    )
    # Put first-layer elbows in [0,1] like before
    w0 = list(net.parameters())[0]        # 0.weight
    b0 = list(net.parameters())[1]        # 0.bias
    elbows = np.sort(np.random.ra(width))
    new_biases = -elbows * to_numpy(w0).ravel()
```

Copy code

```python
    b0.data = to_torch(new_biases)

    # Optional: keep second hidden layer biases small near 0
    # (We won't try to place "elbows" here since inputs are feature ac
    b2 = dict(net.named_parameters())['2.bias']
    nn.init.constant_(b2, 0.0)

    return net


# 2) Slightly generalized gradient-visualization for ANY named paramet
def backward_and_plot_grad(X, model, vis_name='all', title='', legend=
    """
    For each x in X, compute dy/dθ for every parameter θ in `model`.
    If vis_name != 'all', only plot gradients for the chosen parameter
    """
    gradient_collect, vis_collect = {}, {}
    for x in X:
        y = model(to_torch(x).to(device=device))

        # same backprop lines as part (a)
        model.zero_grad()
        y.backward(torch.ones_like(y))
```

```
        for n, p in model.named_parameters():
            # n examples now include: '0.weight','0.bias','2.weight','
            g = p.grad.view(-1).data
            for w_idx, w_grad in enumerate(g):
                key = f'{n}.{w_idx}'
                if key not in gradient_collect:
                    gradient_collect[key] = {'x': [], 'y': []}
                if vis_name == 'all' or vis_name == n:
                    vis_collect[key] = True
                gradient_collect[key]['y'].append(w_grad.item())
                gradient_collect[key]['x'].append(x)

    # Plot (assumes X is sorted)
    for key in vis_collect:
        plt.plot(X, gradient_collect[key]['y'], label=key)

    plt.xlabel('Data Points (X)')
    plt.ylabel(f'Gradient for {vis_name} of {model[0].out_features}-wi
    if legend:
        plt.legend(fontsize=8, ncols=2)
    plt.title(title)
    plt.show()
```

```
# 3) SVD + principal features works unchanged (uses all pa
#    but we provide a convenience wrapper that calls your function.
def compute_svd_plot_features_two_layer(X, model):
    # Reuse your part (b) implementation directly, just pass `model`
    compute_svd_plot_features(X, None, None, None, model)


# 4) Train & visualize for multiple widths (mirrors your one-layer loo
nets_by_size_2layer = {}
widths_2 = widths  # reuse same widths, e.g., [10, 20, 40]

# Hyperparams (you can reuse lr_all, lr_out, n_steps, save_every from
for width in widths_2:
    print('-' * 40)
    print('Two-Layer Width', width)

    net2 = build_two_layer_net(width)
    loss = nn.MSELoss()

    # Optimizer on ALL params (you can tune lr_all for stability)
    opt_all2 = torch.optim.SGD(params=net2.parameters(), lr=lr_all)

    # Save initial weights
    init2 = copy.deepcopy(net2.state_dict())
```

```
    # Train
    hist_all2 = train_network(
        X, y, X_test, y_test,
        net2, optim=opt_all2,
        n_steps=n_steps, save_every=save_every,
        initial_weights=init2, verbose=False, device=device
    )

    nets_by_size_2layer[width] = {
        'net': net2,
        'hist_all': hist_all2,
        'trained_net': net2,
        'init': init2
    }

    # Plot train/test error curves
    plot_test_train_errors(hist_all2)

# 5) Gradient visualizations for BOTH hidden layers + output
for width in widths_2:
    net2 = nets_by_size_2layer[width]['net']

    # First hidden layer params
    backward_and_plot_grad(X, net2, '0.weight', f'Width {width} — L1 W
    backward_and_plot_grad(X, net2, '0.bias',   f'Width {width} — L1 B
```

Copy code

```
# Second hidden layer params                                    Copy code
backward_and_plot_grad(X, net2, '2.weight', f'Width {width} — L2 W
backward_and_plot_grad(X, net2, '2.bias',   f'Width {width} — L2 B

# (Optional) Output layer, too
backward_and_plot_grad(X, net2, '4.weight', f'Width {width} — Outp
backward_and_plot_grad(X, net2, '4.bias',   f'Width {width} — Outp

# SVD + principal features (over ALL params)
print(f"SVD & principal features — Width {width}")
compute_svd_plot_features_two_layer(X, net2)
```

Notes

- Parameter names with this `nn.Sequential` are:
  - First hidden layer: `'0.weight'`, `'0.bias'`
  - Second hidden layer: `'2.weight'`, `'2.bias'`
  - Output layer: `'4.weight'`, `'4.bias'`
- The SVD/principal-features routine already aggregates **all** parameter-feature columns, so it automatically reflects both hidden layers once you pass the 2-layer model.
- If gradients look noisy, consider lower[i] `lr_all` for the 2-layer runs or increasing `save_every` / `n_steps` as needed.

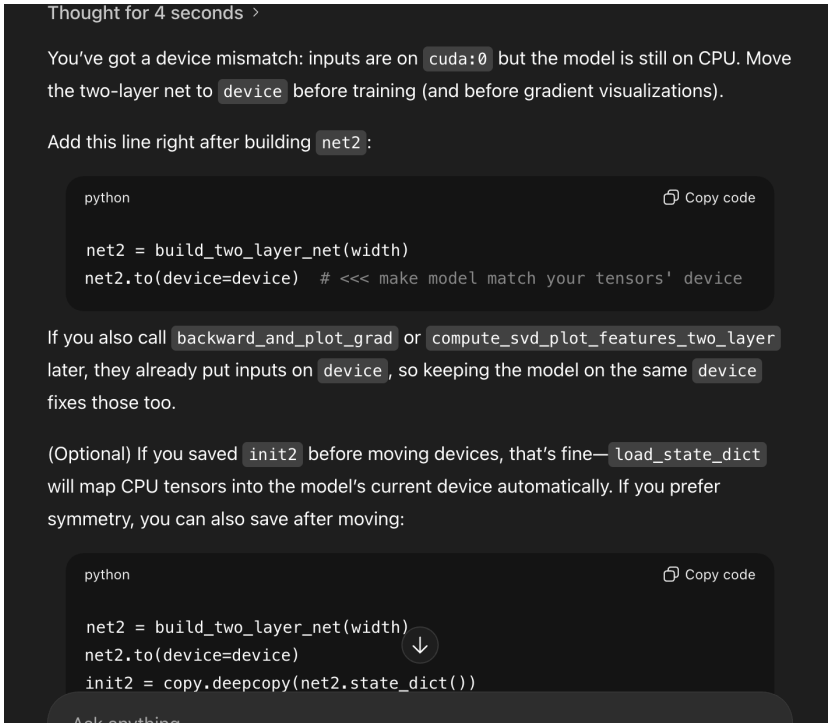
I try the code and get an error, due to mismanaging what device the data is on.

```
...  ----------------------------------------
     Two-Layer Width 10
     ---------------------------------------------------------------------------
     RuntimeError                              Traceback (most recent call last)
     /tmp/ipython-input-762114719.py in <cell line: 0>()
          89
          90     # Train
     ---> 91     hist_all2 = train_network(
          92         X, y, X_test, y_test,
          93         net2, optim=opt_all2,

      ------------------------------------------------------------ 6 frames ------------------------------------------------------------
     /usr/local/lib/python3.12/dist-packages/torch/nn/modules/linear.py in forward(self, input)
         132         Runs the forward pass.
         133         """
     --> 134         return F.linear(input, self.weight, self.bias)
         135
         136     def extra_repr(self) -> str:

     RuntimeError: Expected all tensors to be on the same device, but got mat1 is on cuda:0, different from other tensors on cpu (when checking argument in
     method wrapper_CUDA_addmm)
```

I simply feed the error back to GPT-5 to see what it does with this.

Two-Layer Width 10

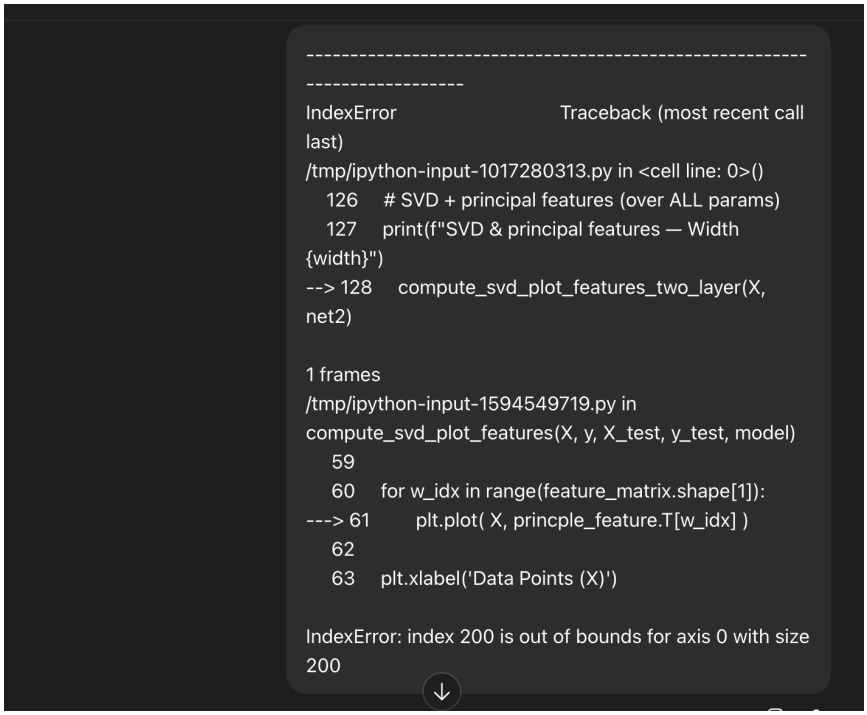
RuntimeError Traceback (most recent call last)
/tmp/ipython-input-762114719.py in <cell line: 0>()
89
90 # Train
---> 91 hist_all2 = train_network(
92 X, y, X_test, y_test,
93 net2, optim=opt_all2,
6 frames
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/linear.py in forward(self, input)
132 Runs the forward pass.
133 """
--> 134 return F.linear(input, self.weight, self.bias)
135
136 def e _repr(self) -> str:

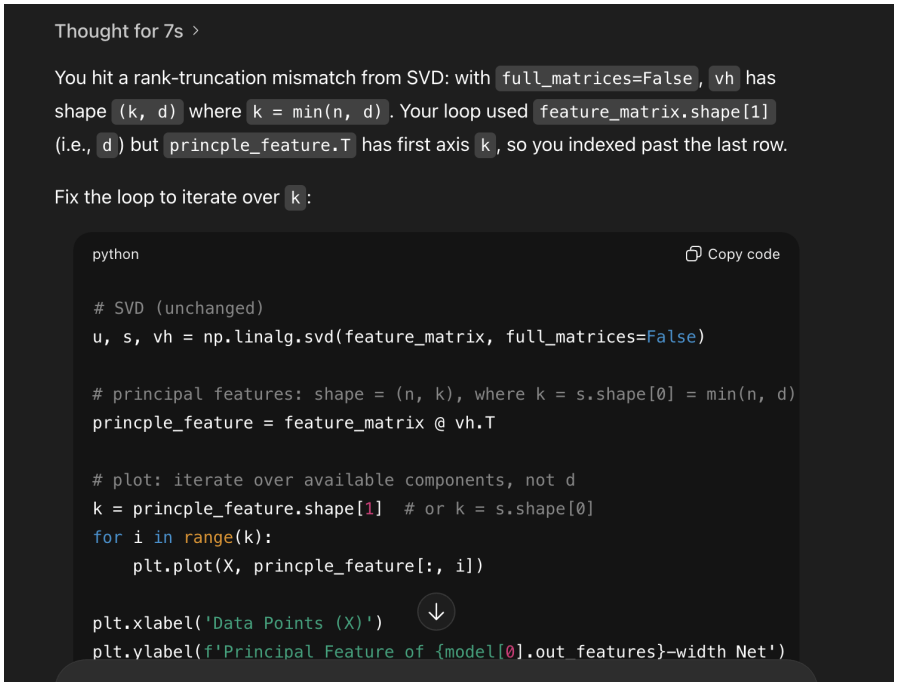

Thought for 4 seconds
You've got a device mismatch: inputs are on cuda:0 but the model is still on CPU. Move the two-layer net to device before training (and before gradient visualizations).
Add this line right after building net2:
python
Copy code
net2 = build_two_layer_net(width)
net2.to(device=device) # <<< make model match your tensors' device
If you also call backward_and_plot_grad or compute_svd_plot_features_two_layer later, they already put inputs on device, so keeping the model on the same device fixes those too.
(Optional) If you saved init2 before moving devices, that's fine— load_state_dict will map CPU tensors into the model's current device automatically. If you prefer symmetry, you can also save after moving:
python
Copy code
net2 = build_two_layer_net(width)
net2.to(device=device)
init2 = copy.deepcopy(net2.state_dict())

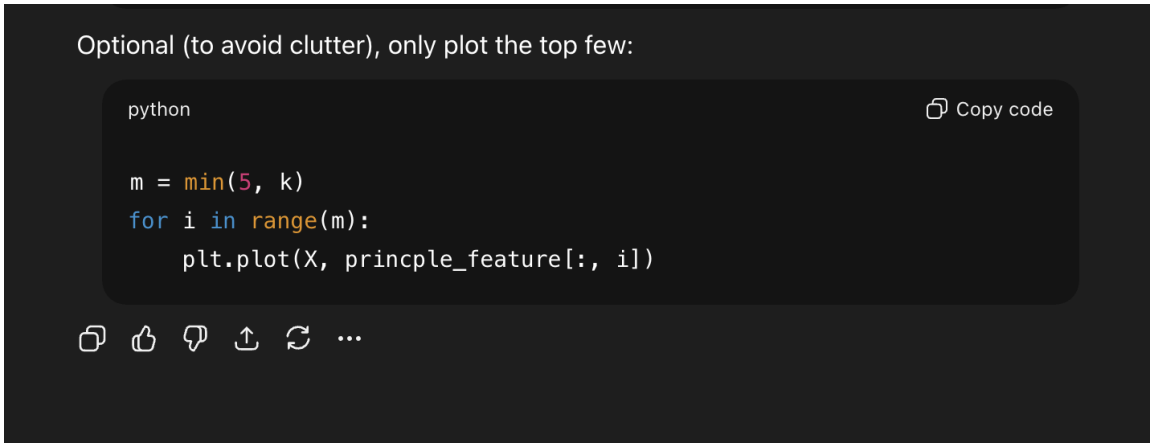
I drop in the fix, adding the to(device) line, and it works mostly. Something interesting to note is that although the correct device is deducible from the context, it didn't consider it on the first attempt.

```
---------------------------------------------------------------------------
IndexError                                Traceback (most recent call last)
/tmp/ipython-input-1017280313.py in <cell line: 0>()
    126     # SVD + principal features (over ALL params)
    127     print(f"SVD & principal features — Width {width}")
--> 128     compute_svd_plot_features_two_layer(X, net2)

                                 1 frames
/tmp/ipython-input-1594549719.py in compute_svd_plot_features(X, y, X_test, y_test, model)
     59
     60     for w_idx in range(feature_matrix.shape[1]):
---> 61         plt.plot( X, princple_feature.T[w_idx] )
     62
     63     plt.xlabel('Data Points (X)')

IndexError: index 200 is out of bounds for axis 0 with size 200
```

I get this index error and just prompt the model with it to see what it does with it.



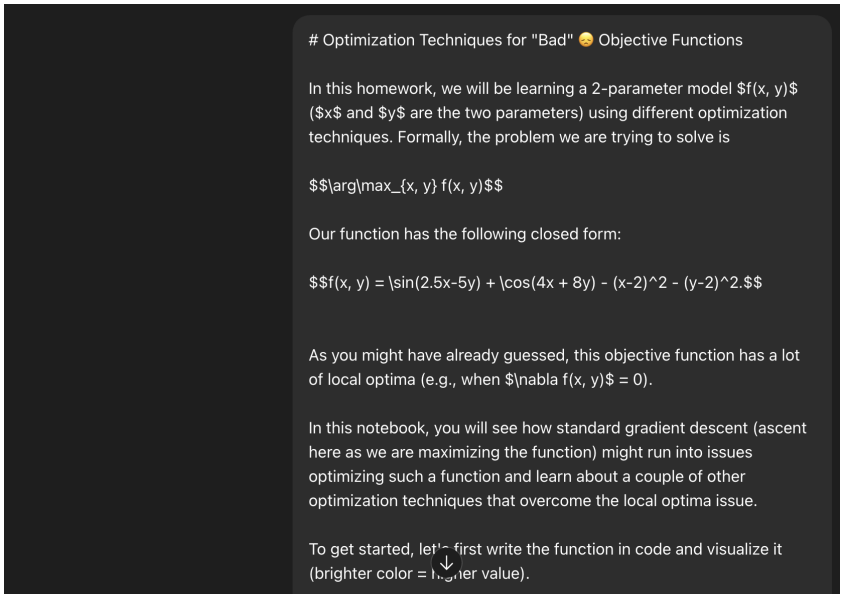




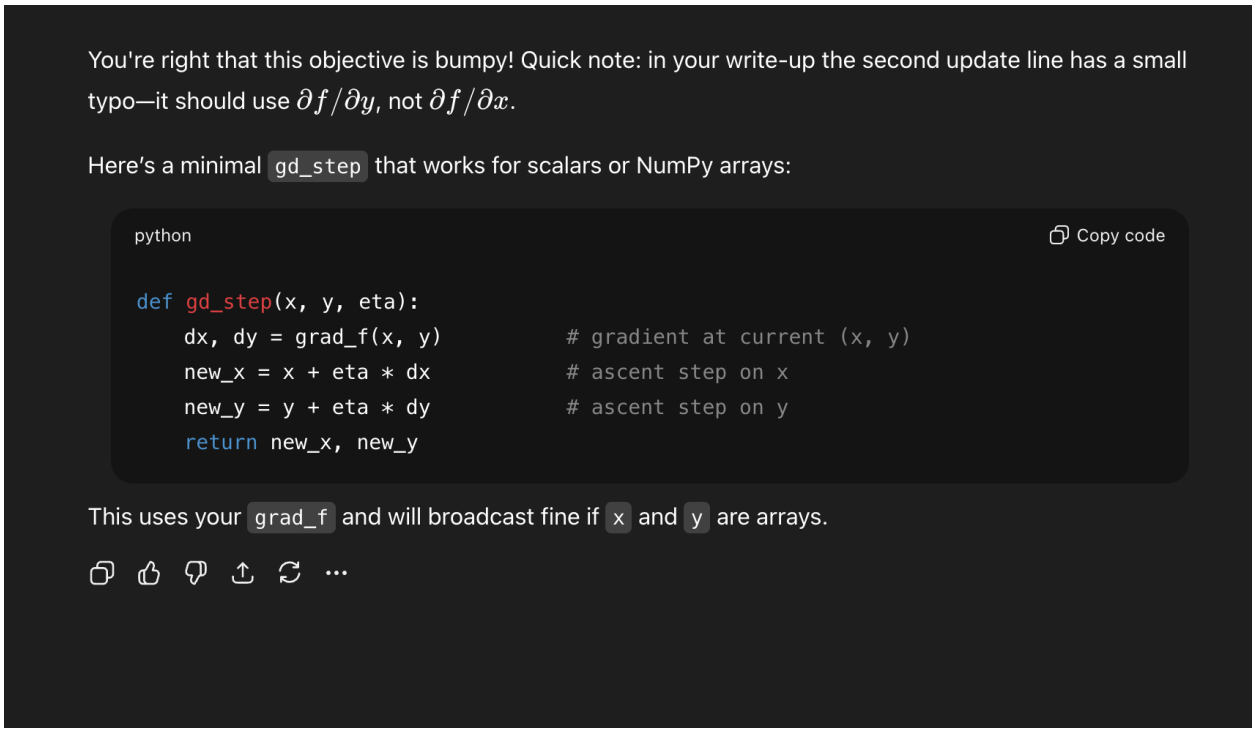
Interestingly, the problem wasn't in the code generated for the last part. It was an error in the previous problem that carried over here. Running with the substitute code, it works perfectly.

6. Optimization Techniques for "Bad" Objective Functions

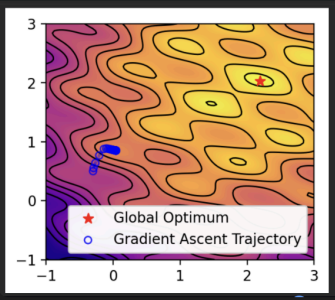
Now I take the same approach of prompting using every cell as context.



It recognizes what code needs to be added without any additional instruction and gives me this:



This code runs perfectly fine and produces the expected result. I try the model on the word question that accompanies this as well:



(a). What do you observe from the optimization trajectory of your (x, y) parameters? Where do your parameters converge to? Try a few different initialization values and see how the convergence changes.

(your answer)

In the next part, we are going to cluster these initialization locations by their values at end of their optimization trajectories.

Thought for 5s >

(a) Observation & convergence

- The ascent path (blue circles) quickly **falls into a nearby basin and converges to a *local* maximizer**, not the global one (red star).
  With your shown init $(x_0, y_0) = (-0.3, 0.5)$, the iterates settle around $\approx (0,\ 0.8)$—a local peak in that valley—while the global peak is up in the upper-right region near $(\sim 2,\ \sim 2)$.
- **Initialization matters a lot.**
  - Inits near the upper-right quadrant (roughly $x, y \in [1.5, 2.5]$) usually climb to the **global optimum**.
  - Inits around the left/center portions (e.g., $(-0.5, 0.5), (0, 0.5), (0.5, 0.7)$) tend to end at **different local optima** like the one you observed.
  - Starting near other bright "islands" in the plot leads to convergence to those islands' peaks—classic **basin-of-attraction** behavior.
- The trajectories typically show **short, curved climbs along the gradient directions**, then **damped steps** as they approach the peak (small gradients near a stationary point). With your step size ($\eta = 0.01$), updates are stable but won't jump basins—so the method can't escape local maxima.

Takeaway: Standard gradient ascent on this highly non-convex, oscillatory surface is **highly sensitive to initialization** and commonly **gets stuck at local optima**. Re-initializing from multiple starting points (random restarts) finds better values and sometimes the global peak.

The model's written answer makes sense. So not only is the model able to finish the code, but it is able to interpret the results effectively.

I continue to the next part of the notebook, using the same strategy of prompting with every cell. Something interesting I do is add image prompts for the different slider settings in the output so the model has some idea of how the slider affects the image.


```
# we put all points that are less than eps apart (Euclidean distance)
together.
def manual_clustering(t_xs, t_ys, max_num_clusters=23, eps=1e-2):
  labels = np.zeros_like(t_xs, dtype=int)

  codebook = []
  while True:
```

Thought for 10s ›

(b) Patterns in the basins (and why):

- The basins appear as **slanted stripes/ellipses** that line up in **two diagonal families**.
  That's because the oscillatory terms depend on the linear combos

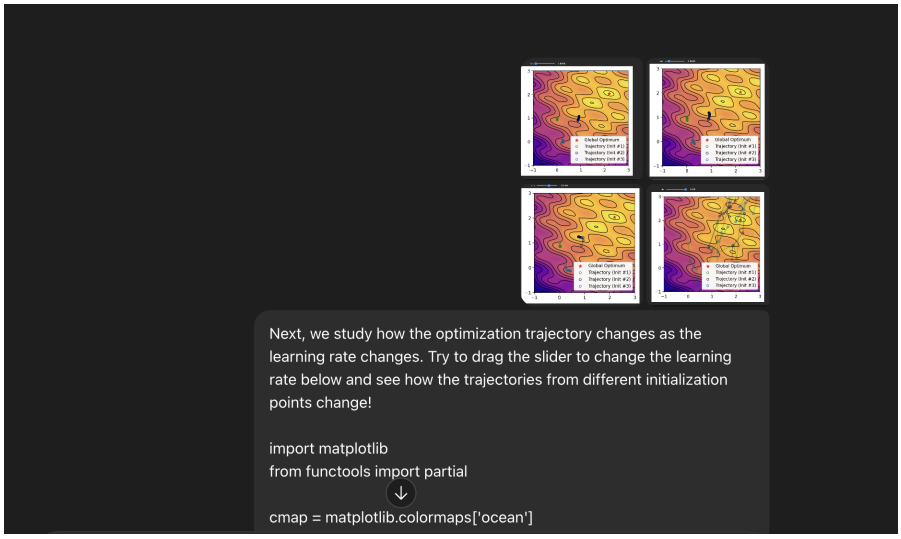
$$u = x - 2y \quad \text{and} \quad v = x + 2y.$$

  Level sets of u are lines $x - 2y = \text{const}$ (slope $+\frac{1}{2}$), and level sets of v are lines $x + 2y = \text{const}$ (slope $-\frac{1}{2}$).
  The plot shows basins elongated along these two directions.
- The **spacing of stripes differs**:
  $\sin(2.5u)$ has period $2\pi/2.5 \approx 2.51$ in $u \Rightarrow$ **wider bands**;
  $\cos(4v)$ has period $2\pi/4 = \pi/2 \approx 1.57$ in $v \Rightarrow$ **tighter bands**.
  Their superposition creates a **lattice / interference pattern**; local maxima occur near favorable phase alignments of both waves, and basin boundaries lie roughly where one term's derivative switches sign (saddle lines).
- The quadratic penalty $-(x-2)^2 - (y-2)^2$ acts like a **bowl centered at** $(2, 2)$, which **tilts the lattice**, enlarging the basins near $(2, 2)$ (the global maximizer) and shrinking those far away.

Net effect: gradient ascent from any point falls into the basin determined by its (u, v) phases—hence the colored regions follow the two sinusoidal directions ↓ their different wavelengths.

GPT-5 again interprets the results correctly. It's able to relate it to the equations above I provided in the context as well.

Proceeding to the next problem, I do the same prompting strategy, adding images with the slider at different settings.



Next, we study how the optimization trajectory changes as the learning rate changes. Try to drag the slider to change the learning rate below and see how the trajectories from different initialization points change!

```
import matplotlib
from functools import partial

cmap = matplotlib.colormaps['ocean']
```

(c) Effect of the learning rate

- **Tiny η (e.g., 10^{-3})**
  Steps are short, trajectories slide along a contour and **settle into the nearest local max**. Super stable, but essentially can't cross the narrow "ridges" separating basins → almost never finds the global optimum unless you start in its basin.
- **Moderate η (a few $\times 10^{-3}$ to 10^{-2})**
  Faster progress; occasionally a step can **hop a shallow saddle** and reach a better basin, but most inits still converge to **nearby** local maxima. This range often gives the **best value-per-time** without exploding, but it doesn't reliably reach the global max.
- **Large η (e.g., 10^{-1})**
  Trajectories **zig-zag/overshoot** and may bounce across several basins. Sometimes you **luck into the global basin**, but behavior is **unstable**—you also see divergence or cycling.

Is there a "good" η for the global optimum?
Not a single fixed one. A slightly larger η increases the chance of escaping a local basin, but too large destabilizes updates. In practice you'd use **multiple random restarts** and/or a **schedule** (e.g., start larger and **decay** η) or add **noise/momentum** to improve odds of reaching the global maximizer.

The model again interprets the provided images correctly and gives a reasonable interpretation.

I now proceed to part 2 with the same strategy of prompting using the markdown and code cells.

Part 2: Smoothing the optimzation problem

We have seen that it is very difficult for gradient ascent to find the global optimum relibly unless the initialization is close to the global optimum. This is due to all the small "hills" and "puddles" that trap the optimization trajectories. Can we optimize a different problem that has a similar global optimum but a much nicer objective landscape?

In this part, we explore a smoothing technique that converts the original objective function into a more smoothed one. In particular, we convolve a 2-D Gaussian kernel over the function ([convolution](https://en.wikipedia.org/wiki/Convolution), [Gaussian kernel](https://en.wikipedia.org/wiki/Gaussian_function)):

$$f_{\sigma}(x, y \mathbb{E}_{\tilde{x} \sim \mathcal{N}(x, \sigma^2), \tilde{y} \sim \mathcal{N}(y, \sigma^2)}\left[f(\tilde{x},

Here's a simple Monte-Carlo estimator for the Gaussian-smoothed objective:

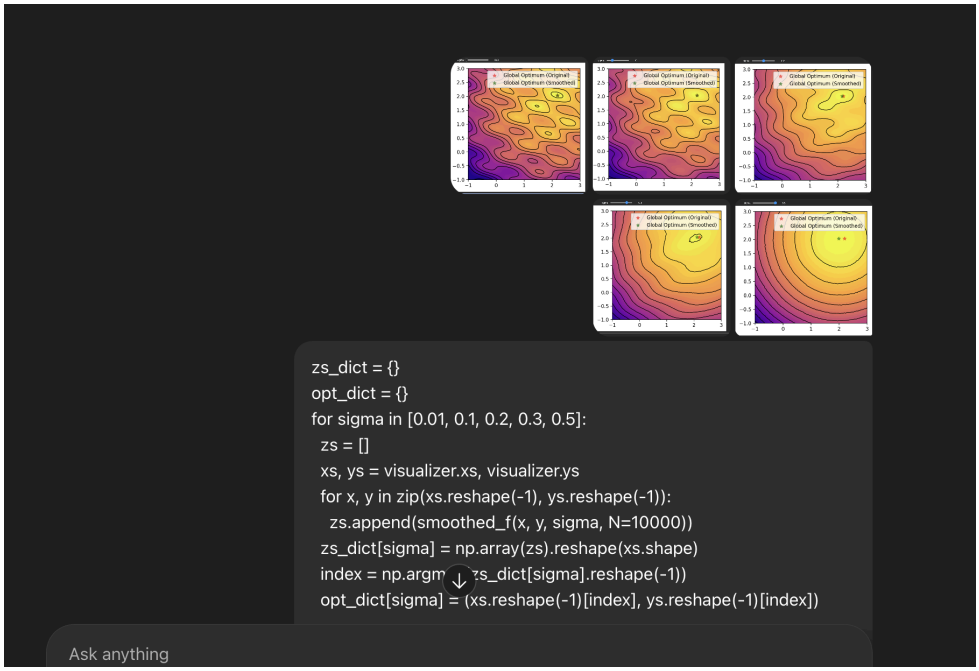
Ask ChatGPT

Copy code

```
def smoothed_f(x, y, sigma, N):
    # Draw N i.i.d. samples around (x, y)
    xs = np.random.normal(loc=x, scale=sigma, size=N)
    ys = np.random.normal(loc=y, scale=sigma, size=N)
    # Evaluate and average
    return float(np.mean(f(xs, ys)))
```

It samples $(\tilde{x}_i, \tilde{y}_i) \sim \mathcal{N}((x, y), \sigma^2 I)$ and returns the empirical mean $\frac{1}{N} \sum_i f(\tilde{x}_i, \tilde{y}_i)$.

Moving onto the text answer part of the problem, I use my strategy of image prompting at the various slider settings:



```
zs_dict = {}
opt_dict = {}
for sigma in [0.01, 0.1, 0.2, 0.3, 0.5]:
  zs = []
  xs, ys = visualizer.xs, visualizer.ys
  for x, y in zip(xs.reshape(-1), ys.reshape(-1)):
    zs.append(smoothed_f(x, y, sigma, N=10000))
  zs_dict[sigma] = np.array(zs).reshape(xs.shape)
  index = np.argm zs_dict[sigma].reshape(-1))
  opt_dict[sigma] = (xs.reshape(-1)[index], ys.reshape(-1)[index])
```

Ask anything

Thought for 6s ›

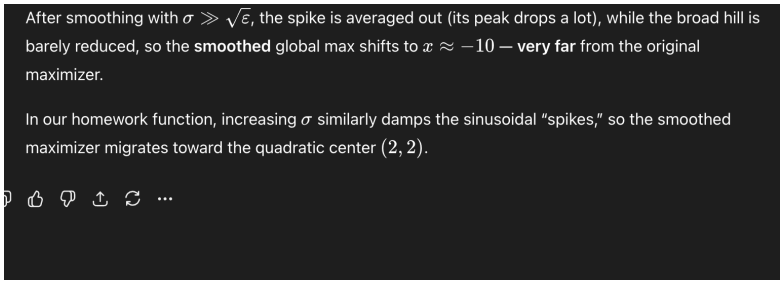
(d) What happens to the global optimum as σ grows (and why)?

- **Observed drift.**
  For small σ (e.g., 0.01) the smoothed optimum (green star) is almost on top of the original one.
  As σ increases, the green star **moves toward the broad bowl center near** $(2, 2)$ and the landscape loses the small ripples. By $\sigma \approx 0.3-0.5$, the ripples are almost gone and the maximizer is essentially that of the quadratic $-(x-2)^2-(y-2)^2$, i.e. $(2, 2)$.
- **Reason.**
  Convolution with a Gaussian is a **low-pass filter**: in the Fourier domain it multiplies each frequency by $\exp(-\frac{1}{2}\sigma^2\|\omega\|^2)$.
  Hence the high-frequency oscillations from $\sin(2.5(x-2y))$ and $\cos(4(x+2y))$ are **heavily attenuated** as σ grows, while the smooth quadratic term remains. The objective therefore becomes smoother and increasingly dominated by the quadratic, pulling the maximizer toward $(2, 2)$.
- **When can the smoothed maximizer be far from the original one?**
  Whenever the original optimum is a **very narrow, sharp spike** (high but tiny support) while a **broader hill** elsewhere has a lower peak but larger average value in a σ-neighborhood.
  Example in 1D (extend to 2D similarly):

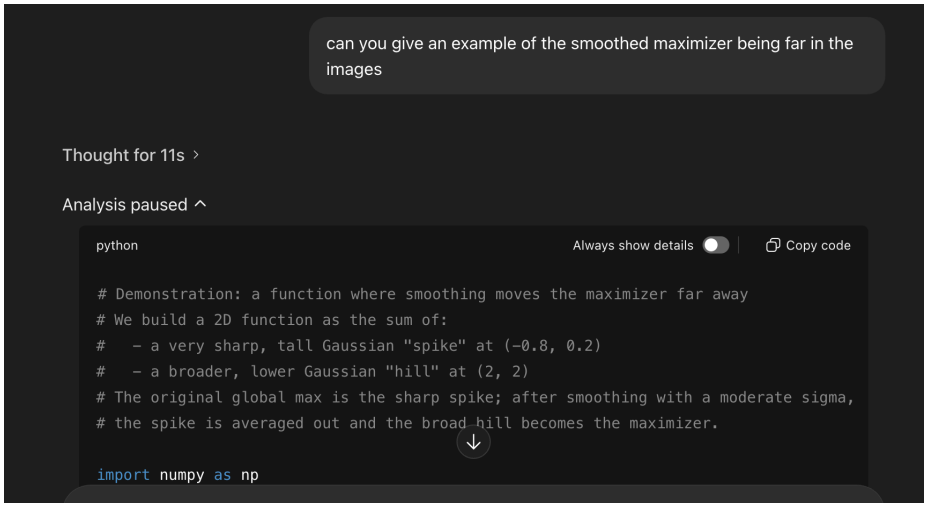
$$f(x) = \underbrace{A\,e^{-(x-10)^2/\varepsilon}}_{\text{very sharp}} + \underbrace{0.9\,e^{-(x+10)^2/100}}_{\text{broad hill}},$$

  with $\varepsilon \ll 1$ and A large so the **original** global max is at $x \approx 10$.

Ask anything



This model is again able to interpret the images I provide and connect them together to make this response. The response is factually correct but it dodges giving an example like asked, so I ask it for one.



It starts building a function to answer this instead of referencing the images so I cut it off.
I now demand that it reference the images.

i mean from the images i provide

Thought for 10s ›

Gotcha—you mean **within your screenshots**.

From the panels you shared:

- **σ = 0.01 → 0.1:** the green "smoothed" star is almost on top of the red one (shift is tiny).
- **σ = 0.2:** there's a small drift of the green star toward the bowl center near $(2, 2)$.
- **σ = 0.3–0.5:** the drift is largest; in your last panel (σ = 0.5) the green star sits a bit left of the red star —roughly a **~0.2–0.3 move in** x and almost the same y. That's the biggest separation visible in your images.

So, in **your images there isn't an example where the smoothed maximizer is "very far"** from the original —because your original maximizer already lies near the quadratic center, and smoothing mainly irons out the sinusoidal ripples without relocating the peak dramatically.

If you want a "far" shift **within this notebook**, you'd need either (i) much larger σ (and a larger plotting window), or (ii) a modified objective where the original max is a narrow spike away from $(2, 2)$. But with the current function and the σ values you plotted, the maximum only moves a short distance.

This analysis is correct.

For part 3a I do the same approach of prompting by copying the cells over.

Our smoothed function successfully removes all the local optimum except the global optimum. However, we don't know how to optimize this smoothed function efficiently yet. The key challenge here is that we actually don't have a good idea on how to differentiate through f_σ or approximate ∇f_σ.

In the following part, we will be exploring three different optimization techniques to address this issue. We are going to use $\sigma=0.3$

Part 3a: Finite-difference method

The first method that we are going to explore is the finite-difference method where we are going to approximate the gradient by trying to step in each dimension of θ and evaluate the difference in the objective function:

$$ \frac{\partial f_{\sigma}(x, y)}{\partial x} \approx \frac{\hat{f}_{\sigma}(x + \delta, y) - \hat{f}_{\sigma}(x, y)}{\delta}$$

Thought for 5s ›

Here's a forward finite-difference ascent step that reuses the **same MC samples** (common random numbers) to reduce variance:

python Copy code

```python
def finite_diff_grad_step(x, y, delta, N, eta, sigma):
    # Draw shared samples around (x, y)
    tx = np.random.normal(x, sigma, size=N)
    ty = np.random.normal(y, sigma, size=N)

    # Monte-Carlo estimates using common samples
    f_xy      = np.mean(f(tx,         ty))
    f_xplus   = np.mean(f(tx + delta, ty))     # ≈ fσ(x+δ, y)
    f_yplus   = np.mean(f(tx,         ty + delta))  # ≈ fσ(x, y+δ)

    # Finite-difference gradient estimates
    dfdx = (f_xplus - f_xy) / delta
    dfdy = (f_yplus - f_xy) / delta

    # Gradient ascent step
    new_x = x + eta * dfdx
    new_y = y + eta * dfdy
    return new_x, new_y
```

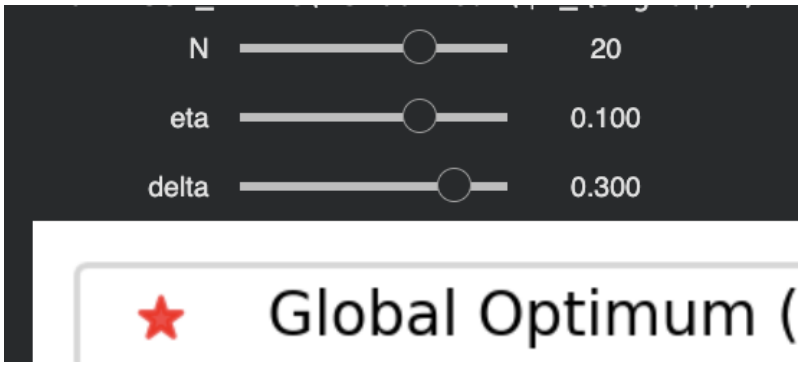
Ask anything

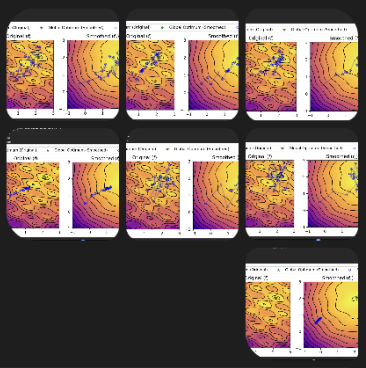
+ Thinking

This follows your definition and works for the smoothed objective f_σ with $\sigma = 0.3$.

This gives me the correct results.

The next subproblem is tricky to prompt since I would have to take a combinatorial approach in my observations to make a judgment. To do this, I choose random slider values and feed the model the images.





Now, we are going to look at how changing η, δ and N influences the learning trajectory. Run the following cell and drag the sliders to try it out yourself!

def get_interactive(grad_step_fn, label_name, init_x=-0.3, init_y=0.5,

thing

(e) What do you observe when you change δ, η (learning rate) and N (the sample size for Monte-Carlo estimate)?

(your answer)

---

please reference the images to answer the question

Thought for 7s ›

(e) What happens as you vary δ, η, and N (from your screenshots):

- **Sample size N (MC averaging) — variance control**
  - With $N = 2$ (first panel), the FD direction is **very noisy**: the path jitters, zig-zags, and takes indirect hops before circling the smoothed peak.
  - At $N = 10$ (second/third panels) the trajectory becomes **much smoother and straighter**, heading to the smoothed maximizer with fewer detours.
  - Takeaway: **Larger N → lower variance, steadier ascent** (but more function evaluations).
- **Learning rate η — step size/overshoot**
  - **Moderate η (≈0.1–0.16)** (second/third panels): fast progress; small oscillations near the top as it "rings" around the peak.
  - **Very small η (≈0.0025)** (fourth panel): the path **crawls** and effectively **stalls** far from the maximizer—hardly any movement across contours.
  - **Larger η (≈0.25)** (fifth panel): updates **overshoot**, bouncing around the peak and taking longer to settle—even more so when the gradient estimate is noisy.
  - Takeaway: there's a **Goldilocks zone**—too small is slow/stuck; too large leads to oscillation or instability.
- **Finite-difference step δ — bias–variance trade-off**
  - **Moderate δ (≈0.3)** (first/second panels): g↓ients are **stable** and point roughly uphill; good progress.

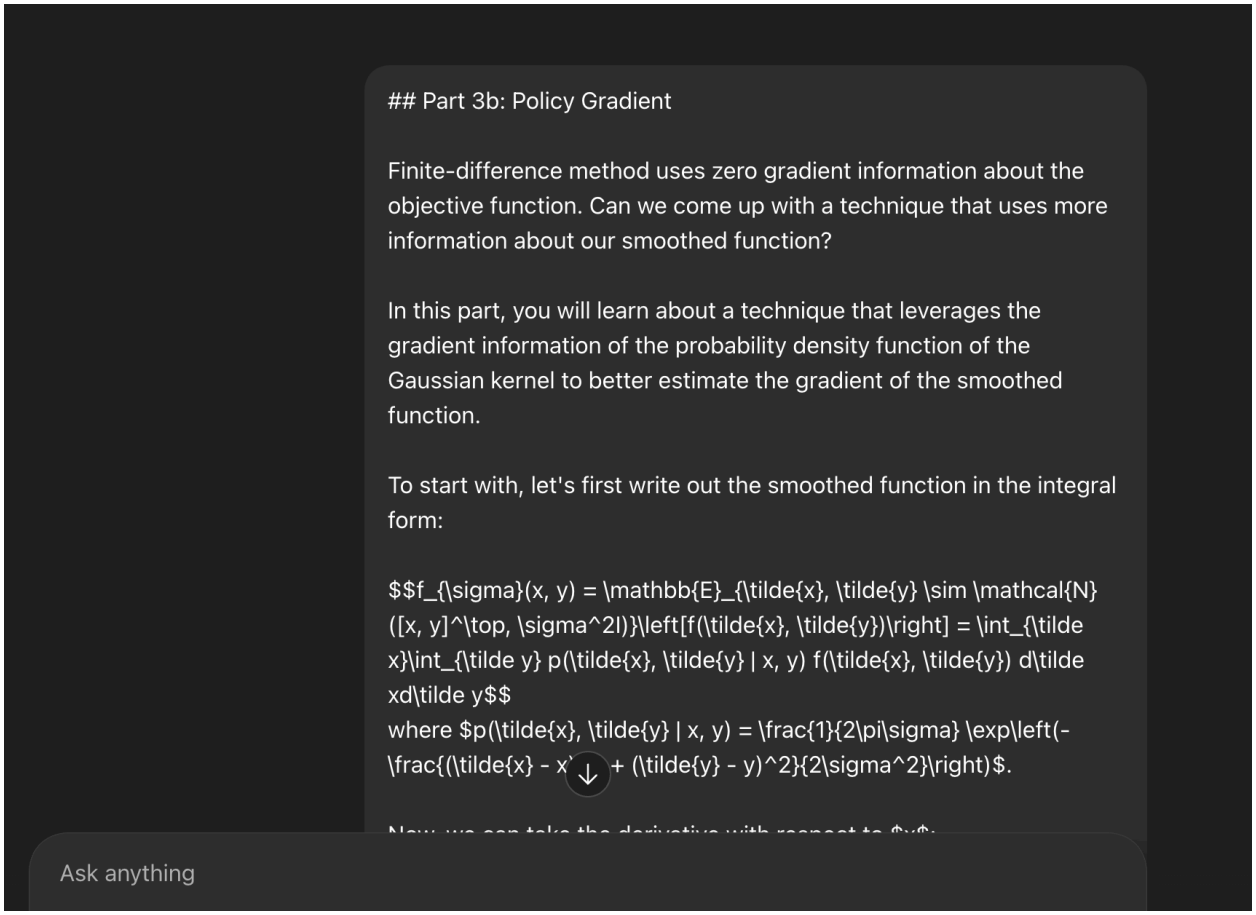
Ask anything

- **Tiny δ (≈0.0025)** (fifth/sixth panels): the FD quotient is dominated by MC noise and numerical error; you see **jittery, erratic steps** around the peak or near-zero movement when paired with a small η.
- **Larger δ** (top row with $\delta = 0.3$ compared to smaller values): gradients are **less noisy** but **more biased** (coarse difference across curvature), causing some misalignment and mild zig-zag near the optimum.
- Takeaway: **δ too small → high variance; δ too big → biased direction.** A **moderate δ (on the order of the smoothing scale σ)** works best.

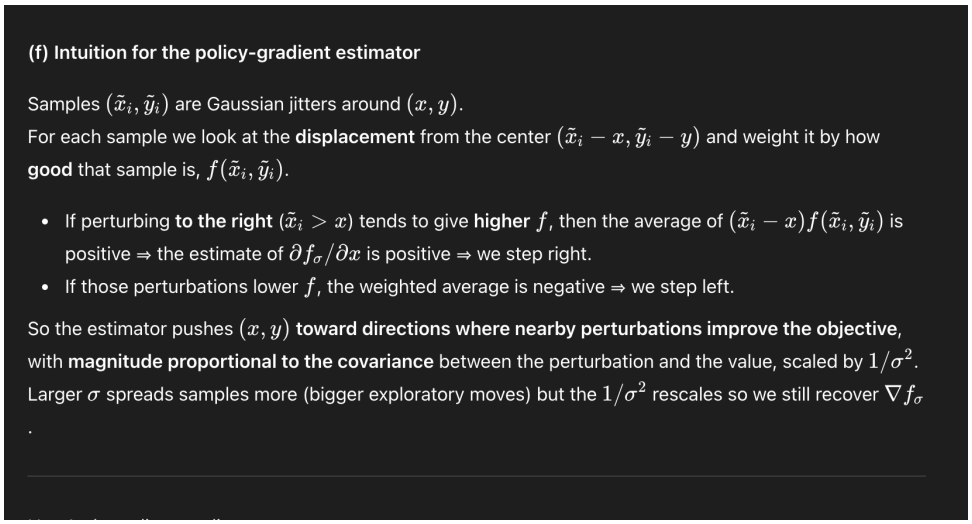
Overall from the images:
Use a **moderate N (≈10–20)**, **moderate δ (≈0.1–0.3, comparable to $\sigma = 0.3$)**, and a **moderate η (≈0.05–0.15)**. This combination gives the **smooth, direct trajectories** that reach the smoothed global optimum in your plots.

Here, the model effectively leverages the images I give to make this analysis. I was surprised somewhat by this since I kind of just chose random output images without really controlling the

settings, but it was able to find patterns. Of course, it's pretrained on this so it might also just be the case that it is saying what it expects to see.



The next problems are a text problem and a coding problem so I just feed in the markdown context and provide the skeleton code.
I didn't actually realize there were two so I prompted both by mistake and got two answers.



I think that this intuitive explanation is correct.

Here's the policy-gradient step:

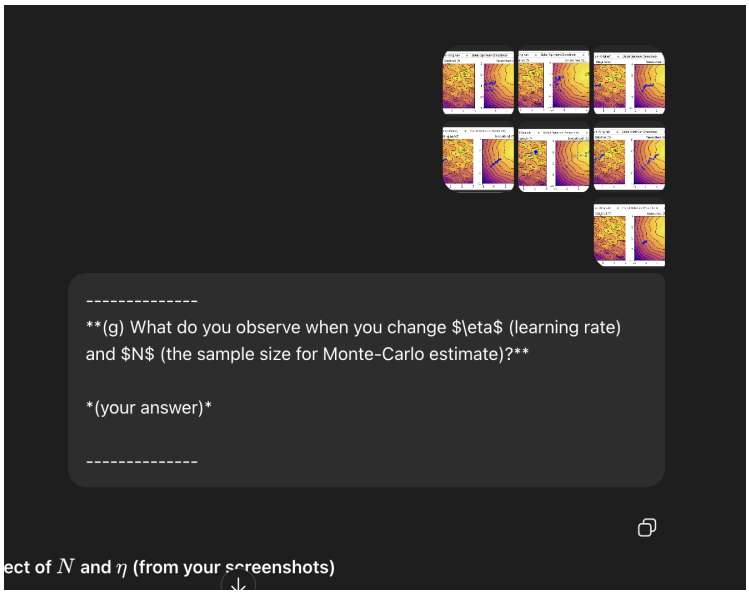
python Copy code

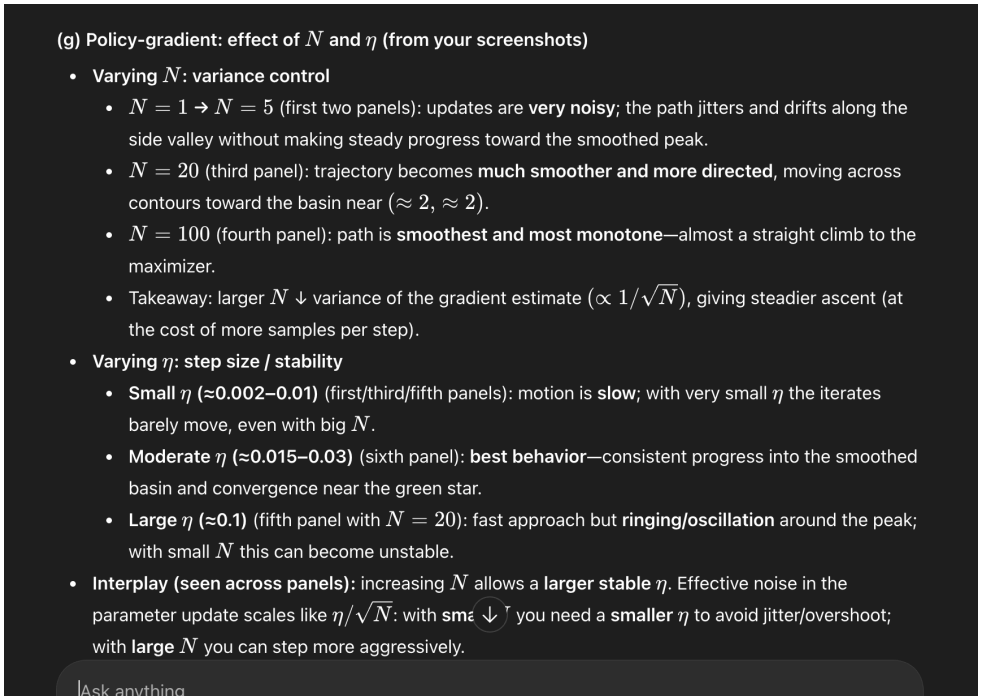
```python
def pg_grad_step(x, y, N, eta, sigma):
    # Sample perturbations around current params
    tx = np.random.normal(x, sigma, size=N)
    ty = np.random.normal(y, sigma, size=N)

    # Evaluate objective at the samples
    vals = f(tx, ty)

    # Policy-gradient / score-function estimates
    dfdx = np.mean((tx - x) * vals) / (sigma ** 2)
    dfdy = np.mean((ty - y) * vals) / (sigma ** 2)

    # Ascent update
    new_x = x + eta * dfdx
    new_y = y + eta * dfdy
    return new_x, new_y
```

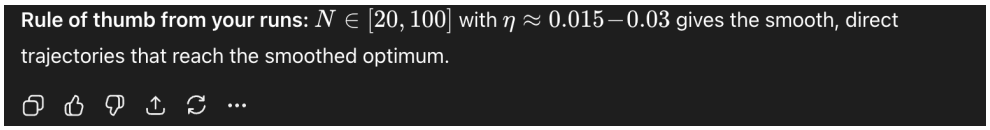
Tip (optional, for variance reduction): subtract a baseline, e.g. $b = \text{mean}(vals)$, inside the products $(tx - x)(vals - b)$ and $(ty - y)(vals - b)$. This leaves the expectation unchanged but often makes trajectories smoother.

This code works and generates the correct output.

The next part is again a question that asks me to vary the slider and interpret a pattern. So I use my image prompting strategy again.







This analysis reflects the images again, and it directly refers to features in the images so it is leveraging the images effectively to get the correct result.

The next problem (3c) is a coding problem so I prompt w/ the markdown and code skeleton again.

Part 3c: Reparameterization Gradient

Policy gradient uses the information about the smoothing kernel to provide a better gradient estimation. Can we use any gradient information about $f(x)$? This might seem a bit unintuitive, as the function that has lots of local optima probably have poor gradient information locally. As we will show in this part, we can actually harness the poor gradient information to provide better gradient estimation of the smoothed function.

Let's go back to the definition of the smoothed function again (and take the gradient operator with respect to $[x, y]$):
$$\nabla f_\sigma(x, y) = \nabla \mathbb{E}_{\tilde x, \tilde y \sim \mathcal{N}([x, y], \sigma^2 I)}\left[f(\tilde x, \tilde y)\right]$$

It would be so nice if we could move the gradient inside the expectation, but that is not possible immediately because the expectation distribution depends on x and y and the term inside the expectation does not directly depend on x and y. Can we somehow rewrite expectation such that the distribution does not depend on x and y? Or in another word, can we separate out the

Here's a simple reparameterization-gradient ascent step (uses your `grad_f`):

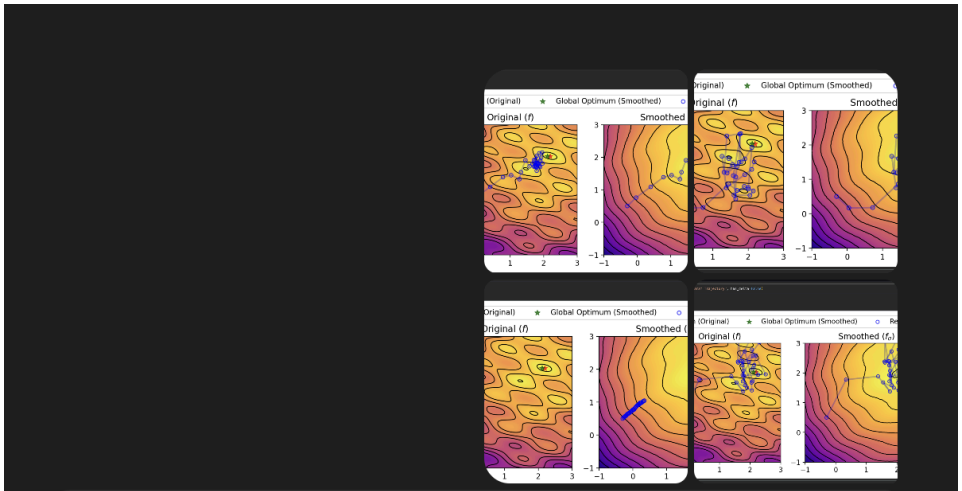
python Copy code

```python
def reparam_grad_step(x, y, N, eta, sigma):
    # Samples:  ẑ ~ N(0, σ^2)  ⇒  (x+ẑx, y+ẑy) ~ N((x,y), σ^2 I)
    tx = np.random.normal(x, sigma, size=N)
    ty = np.random.normal(y, sigma, size=N)

    # Evaluate ∇f at the noisy points and average (MC estimate of ∇f_σ)
    dx, dy = grad_f(tx, ty)          # arrays
    dfdx_hat = np.mean(dx)
    dfdy_hat = np.mean(dy)

    # Gradient ascent step on (x, y)
    new_x = x + eta * dfdx_hat
    new_y = y + eta * dfdy_hat
    return new_x, new_y
```

Notes:

- This directly estimates $\nabla f_\sigma(x, y) = \mathbb{E}[\nabla f(x + z_x, y + z_y)]$.
- In your runner, switch the update loop to call `reparam_grad_step` (it currently calls `pg_grad_step`).

The code works and gives the correct result. I use the same image prompt approach to address the slider in the next question:



(h) Compare the trajectory of reparam gradient vs. policy gradient. What do you observe? Is one more straight than the other one? Why do you think that is the case?

(your answer)

Thought for 4 seconds ›

(h) Reparam vs. policy-gradient (reading your plots)

- In your **reparameterization** panels (e.g., $N = 20,\ \eta = 0.1$; first image), the path climbs **fairly straight** into the smoothed basin and then makes a tight cluster near the green star. When η is pushed high (last image) you see some **overshoot near the top**, but the approach is still comparatively directed. With very small η (third image) it moves slowly but remains **coherent**.
- In the **policy-gradient** panels you showed earlier, for the same N and similar η, the path is **more jagged and meandering**; with small N it jitters a lot, and even with $N = 20$ it takes a **curvier route** and rings more around the optimum before settling.

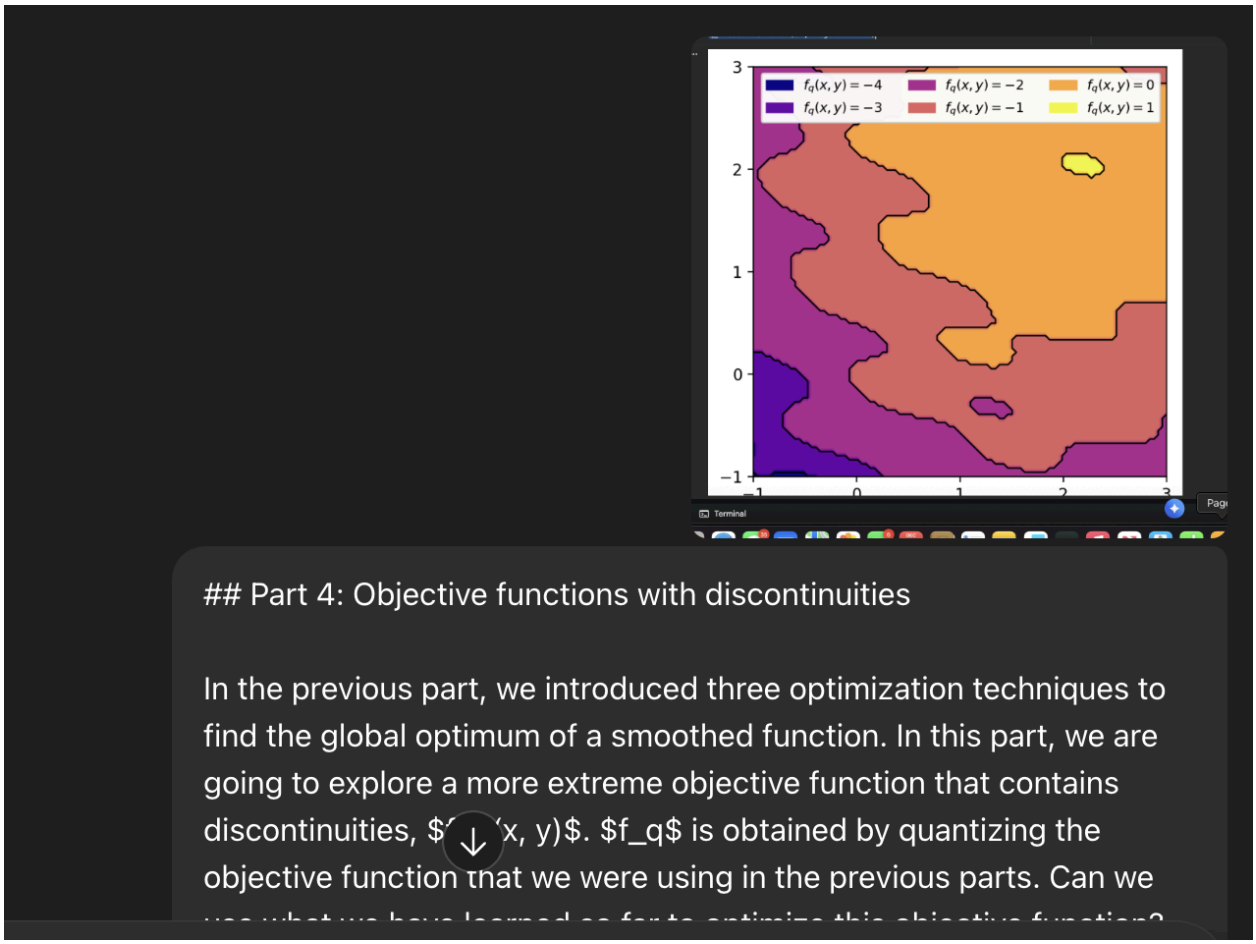
Why reparam looks straighter:
Both estimators are unbiased for ∇f_σ, but their **variance** is different.

- **Policy gradient (score function)** estimates ∇f_σ from terms like $(\tilde{x} - x) f(\tilde{x}, \tilde{y})/\sigma^2$. This multiplies the **noisy function value** by the perturbation, so the variance scales with $\mathrm{Var}[f]/\sigma^2$. When f oscillates, this is high → **noisy directions**, hence the wiggly, indirect trajectories in your images.
- **Reparameterization** averages **true gradients** ∇f at perturbed points: $\nabla f_\sigma(x, y) = \mathbb{E}[\nabla f(x + z_x, y + z_y)]$. The variance depends on $\mathrm{Var}[\nabla f]$, which is typically **smaller** than $\mathrm{Var}[f]$ on a smoothed landscape. That yields **lower-variance steps** → the **straighter, more monotone** paths you observe.

So, in your screenshots, the **reparam trajectory is generally straighter and more sample-efficient**; policy gradient needs **larger** N (or smaller η) to achieve comparable smoothness.

These responses are correct and effectively leverage the context from the images I provided for the past problem.

For part 4 of the notebook, it concerns an image so I provide an image prompt:



```
q_visualizer = Visualizer(qf, n_contour_lines=5, quantized_values=
[-4, -3, -2, -1, 0, 1])

fig, ax = plt.subplots(1, 1, figsize=(4.70, 4.70), dpi=200)
q_visualizer.draw(ax, legend=False)
```

From the visualization, the function only outputs values from $\{-4, -3, -2, -1, 0, 1\}$ with $f_q(x, y) = 1$ being the maximum value (located top right, yellow region).

(i) How well do you expect naive gradient descent algorithm to perform on this quantized objective function? Explain Why.

(your answer)

(j) Can you use reparameterization gradient for this function? Explain why. What about finite-difference?

(your answer)

Thought for 7s ›

(i) Pretty poorly. From your plot the surface is **piecewise-constant** (levels in {-4,...,1}) with **jump discontinuities** on the region boundaries. A naïve gradient method sees

- **zero gradient almost everywhere** (flat regions) → it **doesn't move**; and
- **undefined/infinite gradient on boundaries**, which you almost never hit exactly.
  So, unless the initialization happens to lie near a boundary and your steps cross it, plain gradient descent/ascent will stall at the starting plateau.

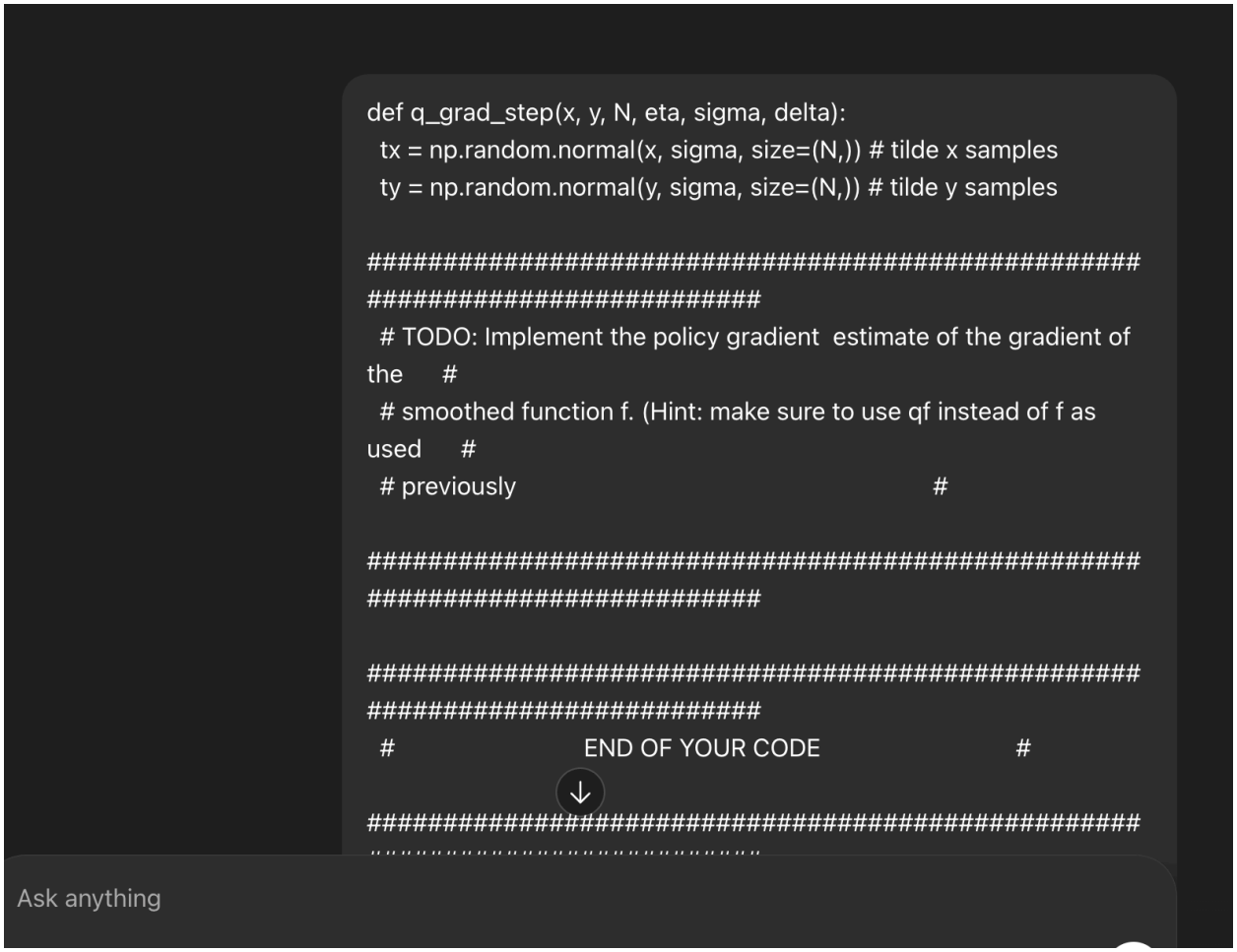
(j)

- **Reparameterization gradient: No** (not useful here). The reparam estimator needs $\nabla f(\tilde{x}, \tilde{y})$. For the quantized f_q, the derivative is **zero almost everywhere and undefined on the jumps**, so the MC average of ∇f_q is essentially **zero**, yielding no learning. (Formally, the identity $\nabla f_\sigma(x) = \mathbb{E}[\nabla f(x+z)]$ relies on f being differentiable a.e.; with pure quantization you get a distributional derivative with impulses you won't capture by point sampling.)
- **Finite difference: Yes, with care—preferably on the *smoothed* objective** $(f_q)_\sigma$. FD only needs function **values**, not differentiability, so if you estimate

$$\frac{\hat{f}_\sigma(x+\delta u) - \hat{f}_\sigma(x,y)}{\delta}$$

It answers the questions correctly and I don't see any hallucinations or misconceptions.

There is a coding question w/ skeleton code for the last problem, so I just copy over the markdown and skeleton code.



```
def q_grad_step(x, y, N, eta, sigma, delta):
  tx = np.random.normal(x, sigma, size=(N,)) # tilde x samples
  ty = np.random.normal(y, sigma, size=(N,)) # tilde y samples

  ########################################################
  ############################
  # TODO: Implement the policy gradient  estimate of the gradient of
  the     #
  # smoothed function f. (Hint: make sure to use qf instead of f as
  used     #
  # previously                                            #

  ########################################################
  ############################

  ########################################################
  ############################
  #                    END OF YOUR CODE                    #

  ########################################################
```

Ask anything

Use the policy-gradient (score-function) estimator but plug in `qf`:

python Copy code

```python
def q_grad_step(x, y, N, eta, sigma, delta):
    # Gaussian samples around current (x,y)
    tx = np.random.normal(x, sigma, size=N)
    ty = np.random.normal(y, sigma, size=N)

    # Evaluate the QUANTIZED objective at the samples
    vals = qf(tx, ty)

    # Policy-gradient estimates of ∇(f_q)_σ
    dfdx = np.mean((tx - x) * vals) / (sigma**2)
    dfdy = np.mean((ty - y) * vals) / (sigma**2)

    # Ascent step
    new_x = x + eta * dfdx
    new_y = y + eta * dfdy
    return new_x, new_y
```

Training loop (append history for plotting):

python Copy code

Ask anything

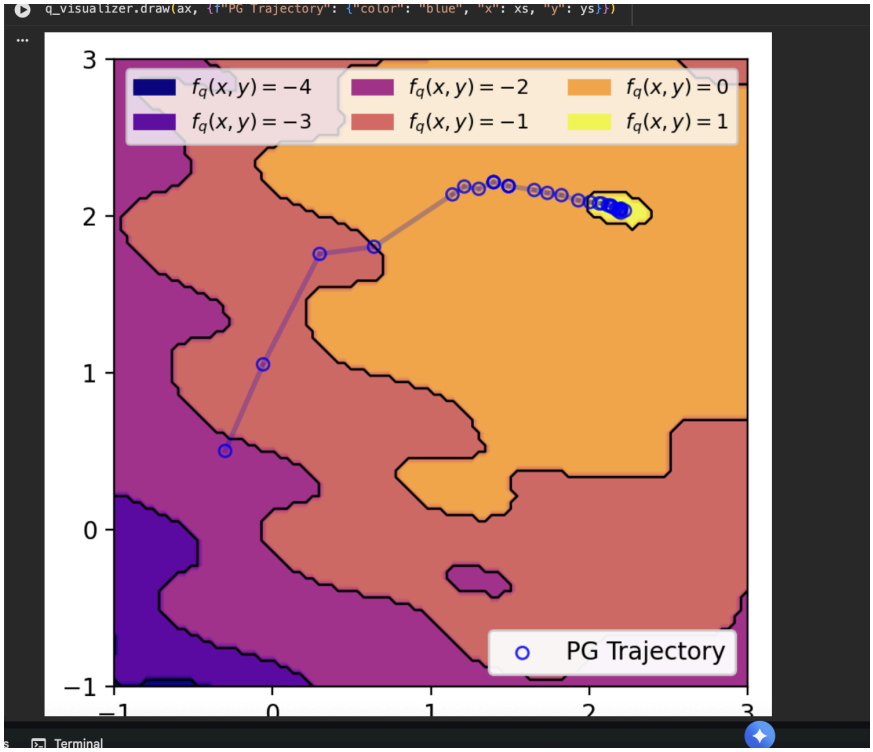
Training loop (append history for plotting):

python Copy code

```python
# history of parameter values
xs = [x]
ys = [y]

for _ in range(num_its):
    x, y = q_grad_step(x, y, N, eta, sigma, delta)
    xs.append(x)
    ys.append(y)
```

This uses the same PG idea as before, but with `qf` values; the Gaussian smoothing (via sampling) provides a useful signal despite the discontinuities.

The code works and I get the following output:



This output is correct, so the model one shotted this without any issue.